University of Plymouth

School of Engineering, Computing, and Mathematics

COMP3000

Computing Project

2025/2026

Rhythmix

Theo Butler

10854606

BSc (Hons) Computer Science (Games Development)

Acknowledgements

I would like to thank my project supervisor Ji-Jian Chin who helped guide me in developing my idea as well as the people who took part in the user testing.

Abstract

This report covers the planning and development process of Rhythmix. This is a rhythm game where players must use a controller and tap the correct buttons at the correct timing using the notes that appear on screen to tell when to press the buttons.

The report starts off with a background on the project which discusses similar games and how Rhythmix is different. As well as this it contains the objectives and deliverables. This is where the requirements for a Minimum Viable Product (MVP) and other optional objectives that would improve the user experience but are not deemed necessary for the MVP.

The next area that is covered is the method of approach. The first part of this section talks about what agile is and how it was implemented in this project. Next is the risk plan. This identifies possible issues that could happen during the course of the project and their effects on preventing the completion of the project. It also contains mitigation strategies that were used to try and reduce the risk of a possible issue or lower the impact it could have on the project. After this is the tools and technologies used. This talks about what was used during the development of the project and discusses the reasoning behind using that particular tool or technology over other options.

Project management is the next section covered by this report which includes how Trello was used to plan sprints, how important version control is and why GitHub was chosen.

Next is the implementation of the project. This goes into detail about how each feature of the game was added in and why it was done this way.

The final parts of the report cover the legal, social, ethical and professional issues the game faces, an end project report, a post-mortem on the achievements and issues of the project and a conclusion evaluating the overall success of the project.

Table of Contents

Acknowledgements

Abstract

Table of Contents

Word Count

GitHub Repository Link

1. Introduction
2. Background, Objectives and Deliverables
    2.1 Background
    2.2 Objectives
    2.3 Deliverables
3. Method of Approach
    3.1 Agile Development Cycle
    3.2 Risk Plan
    3.3 Tools and Technologies
3.3.1 Microsoft Visual Studio Community 2022
3.3.2 Unity Version 6000.2.10f1
3.3.3 PlayFab
3.3.4 Trello
3.3.5 Notepad
4. Project Management
    4.1 Managing Tasks with Trello
    4.2 Version Control
5. Implementation
    5.1 Gameplay
5.1.1 Getting the Notes to Move
5.1.2 Handling Controller Axis vs Button Inputs
5.1.3 Hold Notes and Tap Notes
5.1.4 Gameplay Statistics
    5.2 Pausing the Game
    5.3 Choosing a Song to Play
    5.4 Level Editor
5.4.1 Setup
5.4.2 Timing Lines and Adding Notes

5.4.3 Level Editor Controls
5.4.4 Saving the Level
5.4.5 Loading a Level
5.5 Choosing a Username
5.6 Global Leaderboards
5.7 File Structure for Levels
5.8 Scene Transitions
5.9 Audio Control
6. Testing
7. Legal, Social, Ethical and Professional Issues
8. End Project Report
9. Project post-mortem
10. Conclusion
11. References
12. Appendices
12.1 Appendix A - User Guide
12.2 Appendix B – Trello Board
12.2.1 Epics and User Stories
12.2.2 Sprint Plans
12.3 Appendix C – Testing Results

Word Count: 8881

GitHub Repository Link: https://github.com/Thefudgeman/COMP3000

1. Introduction

This report contains the process of creating a prototype of Rhythmix. This is a rhythm game played on controller where players have to time their inputs to the notes on the screen. The goal of the project was to create a level with global leaderboards where players can compete against each other and create their own levels with their own music and a leaderboard. The main goal of the game is to get the highest score you can get on a level to get a higher position on the leaderboard for that level.

2. Background, Objectives and Deliverables

2.1 Background

The inspiration for this game came from the arcade game Maimai where the screen is a large circle with eight buttons around it that the player must press when the notes approach the hit area (the place that the notes must reach before the button is pressed). However, with this game being limited to arcade machines and not many of them around lots of people are unable to experience the game. Although there is an unofficial version that can be run on a PC it requires a touchscreen monitor which many people do not have and comes with the risk of damaging the monitor if someone is playing a fast or difficult level.

The Persona Dancing games share the similarity of having a circular play area where notes spawn in the centre of the play area and move outwards towards the hit area. Unlike Maimai the Persona Dancing games are on PlayStation which allows a larger audience to play them, and they also have a story that ties into the corresponding persona main line game. They also have different costumes for the characters which the player can equip that will make the gameplay easier or harder.

This project will deliver a complete game made in Unity that runs on Windows. The final build will not contain any errors that cause the game to crash or severely affect the players experience in any way. The game should be able to run on low end systems that have a dedicated graphics card with a target of at least 60 frames per second on capable computers. Instead of having one large circle as the play area like the previous examples there will be a left and a right circle with the left circle using the controllers D-pad inputs and the right one using the ABXY buttons (Xbox controller).

2.2 Objectives

- The player will be able to choose a level to play and use a controller for the inputs during the level
- There should be both tap notes and hold notes in the gameplay
- If a player completes a level, they should see a results screen at the end which displays all the information about the score they just set
- If a player sets a high score the score should be submitted to the global leaderboard for that map
- When choosing a song, the player should be able to press on a level and see the top 25 players on the global leaderboard for that map
- Players should be able to create their own levels in the level editor by adding their own songs and adding in notes where they want
- The level editor should be able to load a level into it and make changes to the level

Optional:

- Be able to navigate the UI with a controller
- Add a mascot in the background of gameplay

2.3 Deliverables

A minimum viable product should include most if not all of the objectives mentioned above and they should not have any issues that severely impact the player's experience. Deliverables for this include:

- Allowing players to play levels using a controller, submit scores to the leaderboard and see the top 25 players on each level
- A level editor where players can add their own songs and add hold and tap notes where they would like

3. Method of Approach

3.1 Agile Development Cycle

This project used an agile development cycle due to its advantages in flexibility and how tasks are broken down into smaller sub tasks instead of one large goal to reach far off into the future. If this project was to be made for a client this would allow me to show them progress every two weeks and keep them informed on what has been done and what will be done next. It would also allow clients some more time to change their mind before committing to an idea as the direction of development can be changed quicker compared to other software development cycles.

Using Trello to manage the sprints meant that I could plan future sprints many weeks before hand and because of the flexibility of an agile approach that meant that future sprint plans could be changed easily without causing any major issues.

3.2 Risk Plan

Potential Risk	How it affects the project	Chance of occurring	Severity	Mitigation Strategy
Corruption of project files	Files are unusable and could mean the entire project must be remade	Low	Severe	Using GitHub for version control means that there is a backup of the project online that can be accessed at any time
Illness	Work may not be able to be completed if the illness is too severe	Likely	Low – Severe	If the illness prevented work from being done, then sprint plans should be reviewed and the work that should have been completed during that time should be spread out

				over the course of the rest of the project
Scope Creep	Too many features to work on could mean many of them remain incomplete or less necessary features get prioritised over more important ones	Likely	Medium	Stick to the original scope and finish everything in it before planning other features that are not necessary for a minimum viable product
Implemented tools do not do what they needed to do	A lot of time could be wasted if a tool that has been implemented is found to not do what is required of it	Low	Medium-Severe	Research should be done before implementing tools to see if they contain all the features required of them
The product does not meet the minimum viable product requirements set	If the requirements set for the minimum viable product are not met, then the product made is not acceptable and the project would be considered a failure	Low	Severe	Plan sprints and spread the workload evenly across the time given to complete the project. Review the work done at the end of each sprint to check on progress throughout the project and adjust how much work is planned for the next sprint if necessary.

3.3 Tools and Technologies

3.3.1 Microsoft Visual Studio Community 2022

Microsoft Visual Studio was used as the code editor due to having precious experience as well as having it already setup with Unity from previous projects. This was chosen over Microsoft Visual Studio Code because to set it up more packages had to be downloaded for Unity and extensions had to be added I Visual Studio Code. I did not want to have any more packages than needed in

Unity as many of them can cause conflicts that take up a lot of time to resolve so choosing not to use it skips all of those problems.

3.3.2 Unity 6000.2.10f1

I chose to use Unity over game engines like Unreal Engine as it is purely a 2D game so creating a project in Unreal Engine would overcomplicate it. Also, Unreal Engine can have very long load times when opening the project or even the executable. Another option was to use Godot but since I have no experience in using it, I would have to learn how to use the game engine for some time before I could start to make good progress on the project.

3.3.3 PlayFab

PlayFab is a service from Microsoft that can handle many things about an online game including player accounts, cloud code, ads, currency and leaderboards. I have used it to create anonymous accounts with a display name and its leaderboards service to allow players to compete against each other on the same level. Another option that could have been used was Unity's leaderboards system, however they do not allow a client to create a new leaderboard and even if you use metadata as a leaderboard ID a player can only have one score on one map. As well as this it has a free plan that has a limited number of API calls. The number of calls and the timeframe differ depending on what API feature you are calling.

3.3.4 Trello

Trello was the tool used to manage the project. This included creating a project backlog for all of the future tasks that needed to be completed, planning current and future sprints and keeping track of progress to ensure that I don't fall behind.

3.3.5 Notepad

Notepad was used when testing and debugging the level editor. This is because the level editor saves information to a text file so using notepad, I can see exactly what was written to the file. I could also make any manual changes I wanted when debugging and testing to ensure the game was reading and writing to it correctly.

4. Project Management

4.1 Managing tasks with Trello

At the beginning of the project some epics and user stories were created to guide the development of the project. As development progressed more epics and user stories were added as new ideas came up which meat that there were goals to aim for once the original epics and user stories were complete as well as influence the development of the current features to prevent having to rework them once work began on the new epics and user stories.

Using Trello I created sprint plans each sprint covering what I would like to complete over the next two weeks. At the beginning of the project a project backlog was created with all the tasks in mind at the time. Using this I could Add tasks from there to the next sprint plan or break them down into smaller tasks to make it more manageable. When tasks are completed, they are marked as complete. When planning the next sprint, tasks should be relevant to each other or build upon previous work instead of being disconnected and unorganised. Images for the full Trello board can be found in Appendix C

4.2 Version Control

Version control was extremely important during the development process due to being able to add small changes, undo and local changes that caused the game to break or even to share it across different devices. If any files were lost or incorrect changes were made using version control allowed me to recover a previous version without any of the unwanted changes added into the project. Since version control also allows you to access it from other devices this was useful to continue working on the project on my laptop or the computers in the university.

For this project I chose to use GitHub since it was something I was familiar with through usage in other modules. The only limitation that caused issues was

GitHub's 50mb commit limit when creating the initial repository for the project as the Unity project was fairly large. Whenever changes were made and appeared to be working in the way that was expected they were committed to the repository so they could be accessed from other devices and as a backup in case the local files were lost.

5. Implementation

5.1 Gameplay

In the gameplay scene the places the notes spawn and move along have been split into eight different lanes. Each lane has its own list of tap notes and hold notes that it must create. I chose to do it this way as then multiple lanes can have a note with the same timestamp and inputs can be checked for each one at the same time instead of checking for inputs one lane at a time. If they were all handled in one script this could create a feeling of input delay especially if the computer has low end hardware. When a level is loaded each lane reads the file that contains the note data and adds the timestamps needed to a list for either tap note or hold notes.

```
void AddNote(string Line)
{
    timeStamps.Add(Convert.ToDouble(Line.Substring(Line.IndexOf(",")+1, Line.Length - 4))/1000);
}

void AddHoldNote(string Line)
{
    HoldNoteData holdNote = new HoldNoteData();
    string[] data = Line.Split(",");
    holdNote.headTime = Convert.ToDouble(data[1])/1000;
    holdNote.tailTime = Convert.ToDouble(data[2].Substring(0,data[2].Length-2))/1000;
    holdTimeStamps.Add(holdNote);
}
```

Figure 1: Code snippet of how notes are added to a lane

Since the level file has all the notes in order by the timestamp that means the lists containing the timestamps are in order as well so when creating the notes for the gameplay the lane starts from the beginning of the list and works its way towards the end. The time the notes are created is the timestamp in the

list minus the time it would take for the notes to move from the centre of the screen to the position that would visually indicate perfect timing to the player. When each lane has no notes left to create the results screen is shown to the player so they can see how well they performed.

5.1.1 Getting the Notes to Move

This is one of the most important parts of the gameplay. In order to get the notes to move I decided to tie their position to the current time of the song. Instead of using Unity's built in function to get the time the song has been playing for I created my own function called GetSongTime. This function uses the number of time samples played and divides is by the frequency of the audio clip which returns a more accurate value than the built in function. Using this I gave it a point that it should reach when the note timestamp and the song time are equal. This point is the correct part of the hit area for the lane that the note was spawned in.

```
float spawnDelay = SongControl.Instance.songDelay - (0.7f / SongControl.Instance.noteSpeed);
timeSinceInstantiated = spawnDelay > 0 && timeInstantiated < 0
    ? (Time.timeSinceLevelLoad - spawnDelay) + timeInstantiated
    : SongControl.GetSongTime() - timeInstantiated;
float t = (float)(timeSinceInstantiated / (0.7f * 2 / SongControl.Instance.noteSpeed));
float sizeScale = timeAlive/(float)timeSinceInstantiated;
```

```
if (laneNumber == 0 || laneNumber == 4 || laneNumber == 2 || laneNumber == 6)
{
    transform.localPosition = Vector3.Lerp(Vector3.up * noteSpawnY, Vector3.up * noteDespawnY, t);
}
else if (laneNumber == 1 || laneNumber == 7)
{
    transform.localPosition = Vector3.Lerp(Vector3.left * noteSpawnX, Vector3.left * noteDespawnX, t);

}
else if (laneNumber == 3 || laneNumber == 5)
{
    transform.localPosition = Vector3.Lerp(Vector3.right * noteSpawnX, Vector3.right * noteDespawnX, t);
}
size.sizeDelta = new Vector2(162.0f/ sizeScale, 557.0f/ sizeScale);
```

Figure 2 + 3: Showing how notes are moved

The size of the note is also tied to the song time. This is because if the notes were their full size when spawned in the centre of one of the circles they would overlap and cause the player to be unable to understand what they were seeing. Using the calculation above the notes reach their full size when they get to the hit area.

5.1.2 Handling Controller Axis vs Button Inputs

With the controls for the gameplay using the D-pad and the XYAB buttons on the Xbox controller that meant that lanes 0-3 had to be treated differently from lanes 4-7. This is because lanes 0-3 use the D-pad for their inputs and the D-pad uses an axis for its inputs instead of normal button inputs. This means that it uses values between -1 and 1 to indicate which direction has been pressed. An input higher than 0 on the X axis means the right D-pad button has been pressed and an on the Y axis it means that the up D-pad button has been pressed. If it is less than 0 that means on the X axis the left D-pad button has been pressed and on the Y axis it means the down D-pad button has been pressed. If the input is equal to zero it means that no button has been pressed.

5.1.3 Hold Notes and Tap Notes

Tap notes were the simpler type to implement as the game only needs to recognise when a button has been pressed. This was done with the Input.GetKeyDown() method in Unity for the ABXY buttons and checking the D-pad axis using Input.GetAxis(). The hit error of the player is the difference between the time the player hit the note and the timestamp of the note which is used for giving the player a score.

```
if (Convert.ToInt32(LaneNumber) > 3)
{
    if (Input.GetKeyDown(input) && SongControl.GetSongTime() - timeStamps[noteHitIndex] > -0.13)
    {
        double hitError = SongControl.GetSongTime() - timeStamps[noteHitIndex];
        Debug.Log(SongControl.GetSongTime() - timeStamps[noteHitIndex] + " error");
        PerformanceManager.Instance.Hit(hitError, Convert.ToInt32(LaneNumber));
        Destroy(notes[noteHitIndex].gameObject);
        noteHitIndex++;
    }
}
```

Figure 4: Showing handling input for XYAB buttons on tap notes

```
if ((timeStamps[noteHitIndex] + 0.13f) <= SongControl.GetSongTime())
{
    PerformanceManager.Instance.Miss(Convert.ToInt32(LaneNumber));
    Destroy(notes[noteHitIndex].gameObject);
    noteHitIndex++;
}
if (axisDown && (Input.GetAxis(input) < 0.3 || Input.GetAxis(input) > 0.3))
{
    axisDown = false;
}
if (Input.GetAxis(input) != 0  && !axisDown && SongControl.GetSongTime() - timeStamps[noteHitIndex] > -0.13)
{
   double hitError = SongControl.GetSongTime() - timeStamps[noteHitIndex];
    PerformanceManager.Instance.Hit(hitError, Convert.ToInt32(LaneNumber));
    ax
    De class PerformanceManager |.gameObject);
    noteHitIndex++;
}
```

Figure 5: Showing handling input for D-pad on tap notes

For hold notes the game checks for when the player presses the button down when the head of the hold note approaches the hit area and when the tail approaches it the game checks for when the player releases that same button. On both the head and the tail the player is given a judgement rewarding them with the appropriate number of points and accuracy. Instead of repeatedly checking if the player is holding down the button the game just checks for the initial press of the button. Then if the player releases the button, it checks their timing for the tail. The head time is checked before doing anything to do with releasing the button as if the player had just it a tap note and released the button then the game would check the tail time of the hold note and destroy it giving the player a miss judgement even though they were not interacting with the hold note. This animation also needs a waiting state where it does nothing and waits to be called. This is because without one the animation would play immediately once the new scene has started which would confuse the player as they have not played any notes yet.

```
if(Input.GetKeyDown(input))
{
    holding = true;
}
if (Input.GetKeyDown(input) && SongControl.GetSongTime() - holdTimeStamps[holdNoteHitIndex].headTime > -0.13 && !headHit)
{
    double hitError = SongControl.GetSongTime() - holdTimeStamps[holdNoteHitIndex].headTime;
    PerformanceManager.Instance.Hit(hitError, Convert.ToInt32(LaneNumber));
    headHit = true;
    holdNotes[holdNoteHitIndex].transform.GetChild(0).gameObject.SetActive(false);
}

if (SongControl.GetSongTime() > holdTimeStamps[holdNoteHitIndex].headTime + 0.13 && !headHit && headMissed)
{
    PerformanceManager.Instance.Miss(Convert.ToInt32(LaneNumber));
    headHit = true;
}
```

```
if (holding && Input.GetKeyUp(input) && holdTimeStamps[holdNoteHitIndex].headTime <= SongControl.GetSongTime())
{
    holding = false;

    if(SongControl.GetSongTime() - holdTimeStamps[holdNoteHitIndex].tailTime >= -0.13)
    {
        double hitError = SongControl.GetSongTime() - holdTimeStamps[holdNoteHitIndex].tailTime;
        PerformanceManager.Instance.Hit(hitError, Convert.ToInt32(LaneNumber));
    }
    else if(SongControl.GetSongTime() - holdTimeStamps[holdNoteHitIndex].tailTime < 0.13)
    {
        PerformanceManager.Instance.Miss(Convert.ToInt32(LaneNumber));
    }

    Destroy(holdNotes[holdNoteHitIndex].gameObject);
    headHit = false;

    headMissed = false;
    holdNoteHitIndex++;

}
```

Figure 6 + 7: Handling hold notes in gameplay

5.1.4 Gameplay Statistics:

When playing the game, the player is given different judgements. These are ‘Perfect!’, ‘Great!’, ‘Ok’ and ‘Miss’. Each of these judgements are based off how accurate the player was to having perfect timing on hitting the note. The player will see each of these judgements during the level as text with the correct word for the judgement appears above the correct circle which also has a different colour depending on the judgement. This text has an animation made with Unity’s animator where the scale of the text increases linearly across a fifty millisecond timeframe and if the animation is not repeated then it will fade away by changing the alpha value of the text.

```
if (hitError <= 0.05)
{
    if(laneNumber < 4)
    {
        leftTimingText.SetText("Perfect!");
        leftTimingText.color = perfectColour;
    }
    else
    {
        rightTimingText.SetText("Perfect!");
        rightTimingText.color = perfectColour;
    }
        score += 1000;
    perfectCount++;
    accuracySum += 100;
    Debug.Log("Perf");
}
else if (hitError > 0.05 && hitError <= 0.1)
{
    if (laneNumber < 4)
    {
        leftTimingText.SetText("Great!");
        leftTimingText.color = greatColour;
    }
    else
    {
        rightTimingText.SetText("Great!");
        rightTimingText.color = greatColour;
    }
    score += 500;
    greatCount++;
    accuracySum += 50;
    Debug.Log("Great");
}
```

Figure 8: Giving the player a hit judgement and score

The score the player gets at the end is used to determine their position on the global leaderboards, the accuracy shows the player how far or close they were to having perfect timing on every note in the level and the max combo is used to tell the player their largest amount of notes they hit I a row without missing.

At the end of the level the player is shown a results screen which fades in when the last notes of the level have been played. This is where they can see all the information about the score they set on the map. This includes their score, accuracy, highest combo and number of each judgement type. The results screen also allows the player to leave the level and return to the song select scene or they can play again. If they choose to play again instead of resetting everything in the scene it just reloads the same scene again using the same information about what level to play.

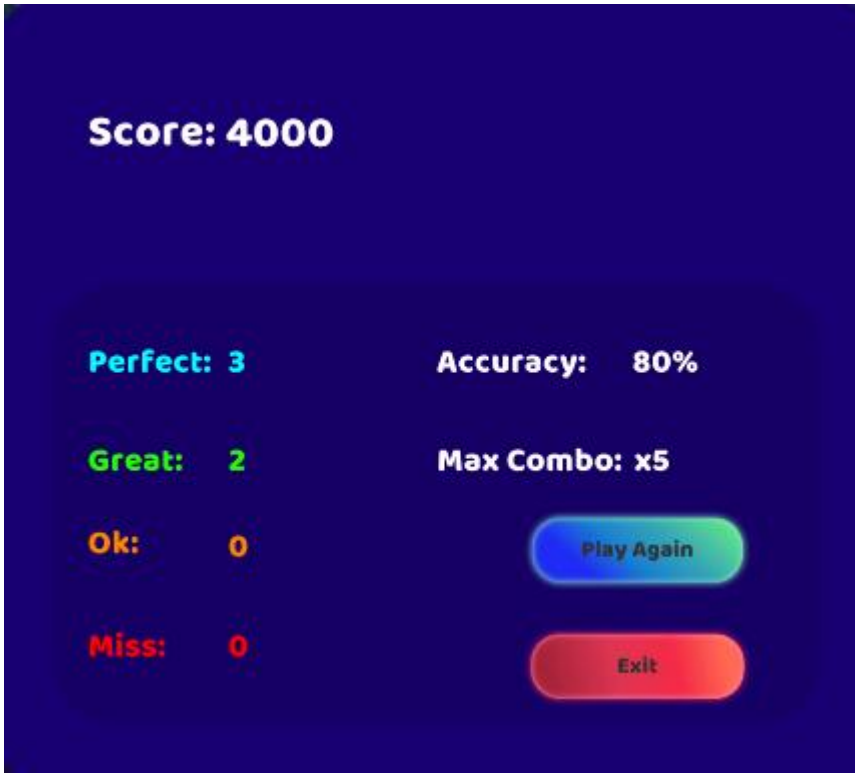


Figure 9: An image of the results UI shown at the end of a level

5.2 Pausing the Game

The game can be paused by pressing escape on the keyboard. When the game is paused all notes stop moving and the song stops playing. Since the position of the notes depends on the song time all that is needed to stop the notes moving is to pause the music. The only exception to this is when the level is paused at the beginning and the timestamp of the notes on screen minus the time it takes to reach the hit area is a negative number. This is because these notes cannot use the song time to determine their position and instead use time since the level has been loaded. By pressing the continue button the song starts playing which causes the notes to start moving again.

5.3 Choosing a Song to Play

In the song select scene if the player has just opened it from the main menu the first song will be selected by default. This will be indicated by the toggle that represents that song being larger than the others in the list. When clicking on a level in the list the game will scroll up or down so that the selected level is

in the middle of the screen. In some cases where this is not possible such as being at the start or end of the list the scroll view scrolls as far as it can so that it does not try to put it in the middle of the screen and then ping back to the top/bottom. This was achieved by multiplying the y position of the toggle pressed by negative one and then subtracting five hundred and two from that value and setting the y position of the content of the scroll view to that value. For the edge cases where this would cause the scroll view to ping around if the value from this calculation was less than zero or greater than one thousand and eighty then its position would be set to either zero or the height of the content minus one thousand and eighty.

```
float height = 256*scrollViewContent.transform.childCount;

if ((transform.localPosition.y * -1) - 542 < 0)
{
    scrollViewContent.transform.localPosition = new Vector3(scrollViewContent.transform.localPosition.x, 0, scrollViewContent.transform.localPosition.z);

}
else if((transform.localPosition.y * -1) - 542 >= height - 1080)
{
    Debug.Log(height);
    scrollViewContent.transform.localPosition = new Vector3(scrollViewContent.transform.localPosition.x, height - 1080, scrollViewContent.transform.localPosition.z);
}
else
{
    scrollViewContent.transform.localPosition = new Vector3(scrollViewContent.transform.localPosition.x, (transform.localPosition.y * -1) - 542, scrollViewContent.transform.localPosition.z);

}
```

Figure 10: Setting the position of the scrollview content

Players can also see the leaderboard of the map they clicked on. This is updated every 0.5 seconds to limit API calls, but this also means that it shows any new scores set by other players shortly after they are submitted to the leaderboard.

When the play button is pressed the name of the song and the sibling index of the toggle are saved in the varsToPass script. The file path to the song is saved so that the gameplay scene knows which files to look for and the sibling index is saved so that when the player returns to the song select scene the level they were just playing is toggled on and in the middle of the screen.

```
public class varsToPass : MonoBehaviour
{
    public string path;
    public static varsToPass Instance;
    public int? scrollviewIndex;
    void Start()
    {
        Instance = this;
    }
}
```

Figure 11: Variables passed between scenes

5.4 Level Editor

When the player opens the level editor for the first time they will be asked if they want to see the instructions on how to use it. This will explain to them what they need to do to setup a level, how to add notes and how to save the level.

5.4.1 Setup

To setup a level the player is shown a screen where they can add a song, choose the BPM of the level and choose the beat division they would like the level to be in. When they add a song the file explorer window limits them to adding mp3 files. This is to means that the file type that they choose does not need to be checked when they are finished. The BPM of the song is needed so that when creating the timing lines the player does not have to find the perfect timing themselves and instead it is given to them. For performance reasons the minimum number of timing lines are created and each timing line can represent different timings as when they reach the end of their lane they are moved back to the centre and are hidden until all the other timing lines have looped around. If it was not done this way tens of thousands of unique timing lines would be needed and that would not be possible to run even on the best of computers.

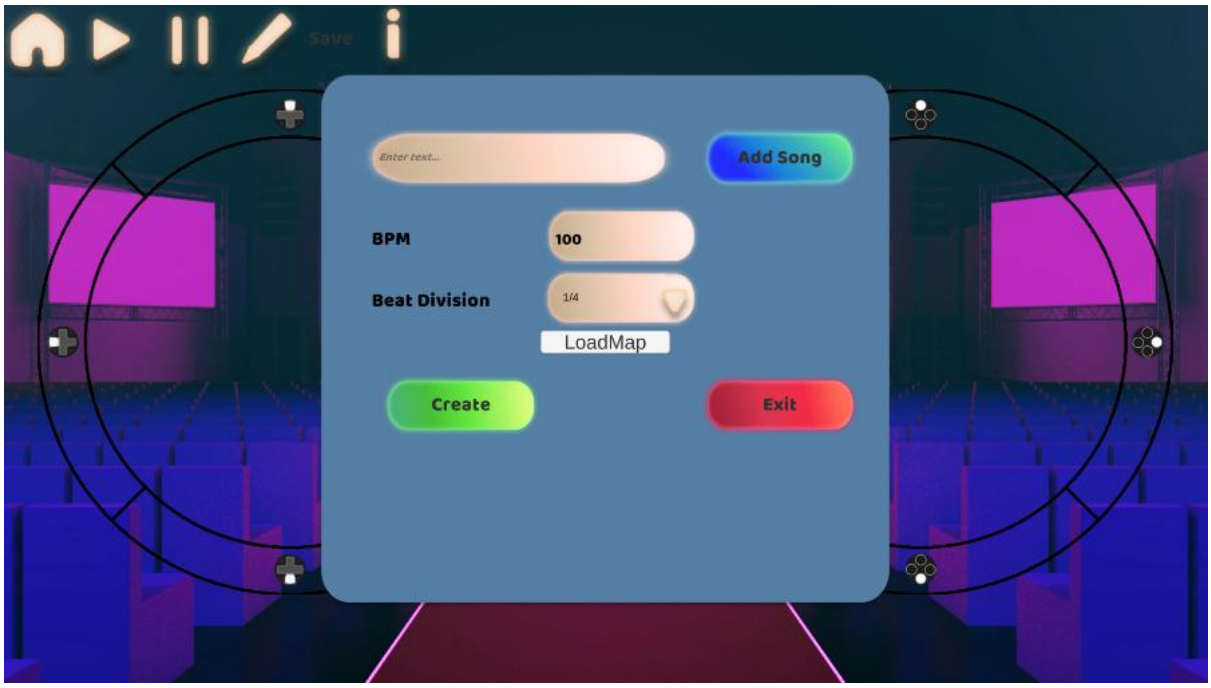


Figure 12: Image of the level editor setup UI

5.4.2 Timing Lines and Adding Notes

When the initial setup has been completed the timing lines will be added to each lane. Each of these timing lines are clickable which allows the player to add the notes. When a note is added to a timing line if it is a tap note the timing line adds its current timestamp to a list for tap notes. If it is a hold note it creates an object from a class called HoldNoteData. This class has both the head time and the tail time of the timestamp, but the player must add both the head and the tail to the level first before the object is added to a list of objects that contain other HoldNoteData objects. If a timing line has a note attached to it and the player clicks it again this destroys the note and removes the timestamp from the list of tap or hold notes. This also removes the problem of having two notes in the same lane having the same timestamp as when the game checks what to do it will add a note if the timing line does have a child or will destroy its child if it has one. Similar to notes in the gameplay scene the hold notes position also depends on the time the song has been playing for, but when they reach the play area they are immediately sent back to the middle and turn invisible so they can be used again.

```
if(!GridUI.Instance.addHoldNote)
{
    if (transform.childCount == 0)
    {
        var newNote = Instantiate(note, transform);
        newNote.transform.position = newNote.transform.parent.transform.position;
        newNote.transform.localPosition -= new Vector3(0, 23);
        newNote.GetComponent<EditorNote>().timeStamp = GetComponentInParent<EditorTimingLine>().timeInstantiated + (0.7f / SongControl.Instance.noteSpeed);
        GetComponentInParent<EditorTimingLine>().noteTimeStamps.Add(newNote.GetComponent<EditorNote>().timeStamp);
    }
    else
    {
        GetComponentInParent<EditorTimingLine>().noteTimeStamps.Remove(transform.GetChild(0).GetComponent<EditorNote>().timeStamp);
        Destroy(this.transform.GetChild(0).gameObject);
    }
}
```

Figure 13: Adding or destroy notes in the level editor

5.4.3 Level Editor Controls

In order to be able to add both tap notes and hold notes the player can press ‘h’ on the keyboard which will allow them to toggle between adding hold notes or adding tap notes. By default it will be toggled off letting the player place tap notes.

At the top of the screen there are a few different buttons. The first one is the home button which lets the player return to the main menu. The next one is the play/pause button. This will stop and start the music in the level editor which will also move the editor timing lines letting the player add notes to later sections of the song. Next is the save button which saves all the changes the

player has made to the level and finally is the info button which gives players a full explanation of these buttons and how to add notes to the level editor.

The player can also press left arrow which will rewind the song by the time difference between timing lines. For example if one timing line has a timestamp of 510ms but the one in front of it has a timestamp of 500ms pressing the left arrow key will rewind the song by 10ms. Similarly, if the right arrow key is pressed it will skip forwards by the same amount of time the left arrow rewinds the song. Because when an audio source in Unity is paused the built in method to get the time the song has been playing will return 0 seconds it instead needs to change the timeSamples property but multiplying the frequency of the audio clip by the difference in time between timing lines.

```
if (Input.GetKeyDown(KeyCode.RightArrow))
{
    SongControl.Instance.audioSource.timeSamples += (int)(SongControl.Instance.audioSource.clip.frequency * (1 / (SongControl.Instance.bpm / 60) / GridUI.Instance.beatDivision));
    Debug.Log((double)SongControl.Instance.audioSource.timeSamples / SongControl.Instance.audioSource.clip.frequency);
}
else if (Input.GetKeyDown(KeyCode.LeftArrow) && SongControl.GetSongTime() > 0)
{
    SongControl.Instance.audioSource.timeSamples -= (int)(SongControl.Instance.audioSource.clip.frequency * (1 / (SongControl.Instance.bpm / 60) / GridUI.Instance.beatDivision));
    Debug.Log((double)SongControl.Instance.audioSource.timeSamples / SongControl.Instance.audioSource.clip.frequency);
}
```

Figure 14: Controls for level editor navigation

5.4.4 Saving the Level

When saving the level a new script runs which loops through all the timing lines and adds all of its tap notes and hold notes into two new lists. Then it sorts the tap notes by timestamp from lowest to highest and the same with the hold notes using the head timestamp. These lists are made out of objects from two new classes. These classes contain the same data from the lists of the timing lines but also include the lane number as well because this is important information needed when loading the notes in the gameplay. Then it writes to the level file but checks if the timestamp of the next note to write from both lists and writes the smallest one. They are written from smallest to highest because in the gameplay scene the notes will be added to the correct lane in the correct order. If for example a note with a timestamp of 1000ms was first in a list that had another timestamp of 500ms after it then the 500ms note would

only be spawned on after the 1000ms note was spawned in. Since notes get destroyed if the time the song has been playing is 130ms larger than the timestamp of the note it would be spawned in immediately be destroyed as well as giving the player a miss judgement. The beat per minute of the song is also written at the end under #MapData which is used when loading a level in the level editor.

```
while (notesIndex < notes.Count && holdNotesIndex < holdNotes.Count)
{
    if ((notes[notesIndex].timeStamp < holdNotes[holdNotesIndex].headTimeStamp))
    {
        sw.WriteLine(notes[notesIndex].laneNumber + "," + notes[notesIndex].timeStamp + ":0");
        notesIndex++;
    }
    else
    {
        sw.WriteLine(holdNotes[holdNotesIndex].laneNumber + "," + holdNotes[holdNotesIndex].headTimeStamp + "," + holdNotes[holdNotesIndex].tailTimeStamp + ":1");
        holdNotesIndex++;
    }
}
```

Figure 15: Notes being written to level file

While saving a pop-up shows the player that the level is saving. Once the level has been saved the Done button that was not interactable is set to interactable so they can close it. This is done to let the player know what is happening and when saving is complete, so they don't lose the changes they made to the level if they leave the level editor before saving is complete.

5.4.5 Loading a Level

Players are also given the choice to load a level so they can edit it. Once the player has chosen the file of the level they would like to edit the timing lines each check all the notes in the file. First checking for the lane number and then seeing if at some point they will have the same timestamp as the note. If they will have the same timestamp at some point it will be added to the timing lines list of timestamp and will be created when the timing lines timestamp is equal to the notes timestamp. This means that players do not have to create a whole level in one go and instead can save their changes and come back another time.

5.5 Choosing a Username

When the game is first loaded there will be an option to choose a username. To do this the player must click on the input field and click on the done button when they have chosen what they would like to be called. Players are automatically logged into the game anonymously, so a request is created for

PlayFab to change their display name. If the request is successful then the display name is updated and the menu to enter the username is closed. Then a string value called ChosenUsername is changed to true in PlayerPrefs. This is used so that once they have chosen a username the menu to choose one is never shown again. If the request is unsuccessful then an error message will be displayed underneath the input field for the username to tell the player why they can't choose the username they want.

```
public void SetName()
{
    var request = new UpdateUserTitleDisplayNameRequest
    {
        DisplayName = username.text
    };

    PlayFabClientAPI.UpdateUserTitleDisplayName(request, OnSetNameSuccess, OnSetNameError);
}

void OnSetNameSuccess(UpdateUserTitleDisplayNameResult result)
{
    Debug.Log(result);
    PlayerPrefs.SetString("ChosenUsername", "true");
    SetUsernameUI.gameObject.SetActive(false);
}

void OnSetNameError(PlayFabError error)
{
    Debug.Log(error.GenerateErrorReport());
    errorText.SetText(error.ErrorMessage);
}
```

Figure 16: Setting name of player

5.6 Global Leaderboards

To add leaderboards to the game I chose to use PlayFab. The first part I had to do was to be able to create leaderboards for each player made level. However, with PlayFab all you have to do is submit a score to a leaderboard using a leaderboard ID and if it does not exist it will create one. This meant that next I had to find a way for player made levels to have a unique ID. This was done by creating a timestamp of when the player who made the level chose to submit their level to be given a leaderboard and storing it in the file containing the level data.

When submitting a score the game must first retrieve any scores the player has made to that level and check if it is higher than the score they just set. If it is higher than the score is not submitted to the leaderboard but if it is the old one will be removed and the new one will be added. Although there is an option on

PlayFab to have it only update scores by highest score for each player this must be done manually for each leaderboard which is not a feasible option should many levels be made.

```
void OnGetPlayerScore(GetPlayerStatisticsResult result)
{
    foreach (var resultScore in result.Statistics)
    {
        if(resultScore.StatisticName == mapID)
        {
            Debug.Log(resultScore.Value);
            playerLeaderboardScore = resultScore.Value;
        }
    }
    Debug.Log(playerLeaderboardScore.ToString());
    if (score > playerLeaderboardScore)
    {
        Debug.Log("Submitting Score");
        var request = new UpdatePlayerStatisticsRequest
        {
            Statistics = new List<StatisticUpdate>
        {
            new StatisticUpdate
            {
                StatisticName = mapID,
                Value = score,

            }
        }
        };
        PlayFabClientAPI.UpdatePlayerStatistics(request, OnScoreSubmitted, OnScoreError);
    }
}
```

Figure 17: Submitting score to global leaderboard

5.7 File Structure for Levels

In order to play levels, I have created text files that store all the information about the level. Below is an example of what it looks like:

1,749.9999:0

4,6150,6350:1

#MapData

BPM:100

LeaderboardID:639111095584340343

The first number of each line represents the lane that the note is to be created in. If the number at the end after the ':' is a 0 that indicates it is a tap note below shows what each number means for a tap note:

laneNumber,hitTime:noteType

Here ‘hitTime’ shows what time the player should press the button that corresponds to the lane it is in. The time is stores in milliseconds.

If the number after the ‘:’ if a 1 then that means it is a hold note. Below shows what each number means for a hold note:

laneNumber,headHitTime,tailHitTime:noteType

‘headHitTime’ refers to the time the player must press down on the button for the corresponding lane and ‘tailHitTime’ refers to when the player must release that button.

Lastly everything after ‘#MapData’ contains other information needed about the map. ‘BPM’ is the beats per minute of the song. This is used in the level editor to create the timing lines that are needed to show where the player can add notes to the level. ‘LeaderboardID’ is the timestamp that the player who created the level submitted it to be added to the leaderboards. This is then used when a player completes a level to submit their score to the global leaderboards.

5.8 Scene Transitions

When changing scene, the game fades out and back in when the new scene has been loaded. This is done similarly to the judgement text where a black square is put over the top of everything on screen and using Unity’s animator the alpha value slowly increases or decreases depending on if the player is entering a new scene or exiting an old scene. A difference between them is that this animation uses something called a trigger which the animation waits for before playing the animation. This animation also does not have to have a waiting state like the one for the judgement text as the first clip to play is the one where the black square fades out revealing the new scene and then waits for the trigger to be set to play the next clip of having the black square fade in to cover up the screen.

5.9 Audio Control

In Unity there is a tool called an audio mixer. This allows you to have different groups to control separate parts of the audio. I chose to have master volume to control all the different audio groups, music to control the volume of the songs that play, soundFX for things such as button presses and hit sounds which affect the volume of the sounds that play when the player misses or hits a note in the gameplay. To use these correctly a different audio source is needed for each audio group.

In the main menu there is the options button. A menu will show up giving the player the option to adjust the volume of each audio group. The way they can change the volume is by adjusting the position of the slider. Volume is measured in decibels but because decibels are measured logarithmically and an increase of ten decibels makes the sound appear twice as loud. Because of this Mathf.Log10 is used so that increasing the value of the slider increases the loudness linearly.

```
    void Start()
    {
        masterVolume.value = PlayerPrefs.GetFloat("MasterVolume");
        musicVolume.value = PlayerPrefs.GetFloat("MusicVolume");
        hitSoundVolume.value = PlayerPrefs.GetFloat("HitSoundVolume");
        soundFxVolume.value = PlayerPrefs.GetFloat("SoundFXVolume");

        audioMixer.SetFloat("Master", Mathf.Log10(masterVolume.value) * 20);
        audioMixer.SetFloat("Music", Mathf.Log10(musicVolume.value) * 20);
        audioMixer.SetFloat("HitSounds", Mathf.Log10(hitSoundVolume.value) * 20);
        audioMixer.SetFloat("SoundFX", Mathf.Log10(soundFxVolume.value) * 20);
    }
}
```

Figure 18: Setting audio volume based off saved settings

When the player is done adjusting the volume the values of the sliders are saved to PlayerPrefs. This means that when the game is closed and reopened the volume the game can get the volume settings they set instead of resetting every time they open the game.

6. Testing

Throughout the project new features were tested to see if they work in different situations to ensure the final product does not have any game breaking bugs.

Towards the end of the project user testing was performed to get feedback on how the game could be improved. Each user who participated was first asked to create a level in the level editor which required them to add a song, enter the beats per minute of that song which was given to them and choose the beat division. They then had to create a level by adding notes and saving their changes. Next, they would have to navigate from the level editor to the song select scene to choose a level they could play. They were given an option between playing the level they just made or playing a premade level that had more notes to play.

Once they were done, they were asked to fill out a form to give feedback on various parts of the game. This included UI navigation, ease of understanding about the level editor, how difficult it was to understand the gameplay and some other questions that are shown in Appendix B.

7. Legal, Ethical, Social and Professional Issues

A legal consideration I must make is the use of music. The music included in the game by default must not contain any copywritten material and I must not provide any way to distribute the music added by players creating levels themselves or encourage it. If I were to distribute the music myself this could lead to the game being taken down and even lawsuits with the affected artists or their music labels.

Another legal consideration is collecting user data. I have chosen not to collect their data and instead use an anonymous sign in through PlayFab to avoid these issues. This means that I only store a playerID which is used for submitting scores to the leaderboard and no other data.

An ethical consideration is not getting players too addicted to the game. Although the point of developing a game is to make something people want to play you should not create a something that causes people to play too much that it affects other aspects of their life.

Another ethical consideration about sharing music is that the artist will not get paid when players listen to a song through the game unlike how they do get paid when their music is streamed through apps like Spotify and YouTube. This also discourages me from providing a place for players to share music besides the possible legal repercussions.

8. End Project Report

This project has met all of the original aims set at the beginning of the project. This includes letting players create a level using their own music, adding both hold notes and tap notes in the gameplay and creating leaderboards for each player made level allowing players to compete against each other on their own maps. Since using a controller wasn't the original idea navigation through the menus is not possible with a controller and must be done with a mouse but this is something that could be fixed with more time. As well as this the level editor has too much going on with the number of timing lines which would make navigation with a controller extremely hard but there does not seem to be a good solution to this given the number of timing lines needed in order to give the player the freedom to place notes at whatever timing they like.

In the gameplay scene players can use the D-pad button on the controller to play the notes that appear in the left play area and can press the ABXY buttons on an Xbox controller to play the notes in the right play area.

Once they have finished a level their score is submitted to the leaderboard for that map and the top twenty-five players scores will appear in the song select scene for the level that is selected.

Players can also go into the level editor to create their own levels. First they must setup the level by adding their own music, entering the beats per minute of the song and choosing the beat division they would like. Once the timing line have been created the player can click on each of them to add notes. By pressing 'h' they can toggle between adding tap notes and adding hold notes. There is a visual glitch with the hold notes where the tail does not disappear when it should and when moving through the song using the arrow keys the hold note will not be recreated.

Once they are done they can press the save button to save the changes to the level they have made and then go to the song select scene to play it. In order

for it to be added to the leaderboard the level must be submitted to the leaderboard by pressing the ‘Submit Map’ button on the main menu.

Due to the difficulty encountered creating the level editor as well as the initial approach of using Unity’s leaderboards service not working none of the optional objectives were achieved. Adding UI navigation with a controller would have been the next task to complete as it would have improved the user experience due to them not having to switch between using a controller for the gameplay and a mouse to navigate the UI. Adding a mascot would also greatly improve the player experience as it would give them something to associate the game with. Also, different aminations could have been played during the gameplay depending on the players performance such as tripping over or appearing nervous when they are doing bad or appearing confident when playing well.

9. Project Post-Mortem

The use of an agile development cycle helped with this project a lot. Using agile meant that work was completed at a steady pace instead of rushing everything towards the end which improved the overall quality of the project. Some of the initial scheduling with the gantt chart ended up being inaccurate mostly due to the implementation of the level editor. Originally the level editor was planned to be completed by early February but with the difficulties encountered such as file permissions, removing hold notes from the list of timestamps, adding custom music and many other smaller issues this delayed the full completion of the level editor by over one month. Although this was delayed, adding other features was much easier which allowed me to finish the project on time.

A mistake made during the planning of adding leaderboards to the game was not doing enough research into Unity’s leaderboards before implementing them. When I was looking at what it was capable of I saw that leaderboards could be created programmatically instead of having to be made manually. This was one of the most important things I was looking for from a leaderboard service as I did not want to create leaderboards for player made levels manually. Once this had been implemented I found that you had to have the admin SDK for Unity’s leaderboards to be able to add leaderboards in the way that I want so this was not a viable option anymore.

10. Conclusion

This project's goal was to create a rhythm game where players can compete against each other and create their own level which have both been achieved making this project a success. The game contains all of the deliverables to a satisfactory level with the time given and gives the players freedom to create what they want. Although I believe the project to be a success there are still some minor issues that I would have liked to be fixed however doing so would require spending more time on it than can be justified for how little it affects the player experience.

11. References

Microsoft Visual Studio Community 2022 (code editor). No longer available. [Online] Microsoft Visual Studio Community 2026 available at https://visualstudio.microsoft.com/downloads/. Last accessed 02/05/2026

Unity Version 6000.2.10f1 (game engine). [Online] Available at https://unity.com/releases/editor/archive/. Last accessed 02/05/2026

PlayFab (backend platform for games). [Online] Available at https://developer.playfab.com/. Last accessed 02/05/2026

Trello (project management tool). [Online] Available at https://trello.com/. Last accessed 02/05/2026

Github (version control tool). [Online] Available at https://github.com/. Last accessed 02/05/2026

Assets used

Alex Ghrol – Energetic Sports Action (music). [Online] Available at https://pixabay.com/music/rock-energetic-action-sport-500409/. Last accessed 02/05/2026

Alan Robinson – GamePad and Keyboard Input Sprites for TextMesh Pro (Unity asset pack). [Online] Available at https://assetstore.unity.com/packages/2d/gui/icons/gamepad-and-keyboard-input-sprites-for-textmesh-pro-310114. Last accessed 02/05/2026

Sky Den Games – Free Casual GUI (Unity asset pack). [Online] https://assetstore.unity.com/packages/2d/gui/free-casual-gui-332804. Last accessed 02/05/2026

Freesound_community (oldedgar) – Pop! (hit sound). [Online] Available at https://pixabay.com/sound-effects/film-special-effects-pop-91931/. Last accessed 02/05/2026

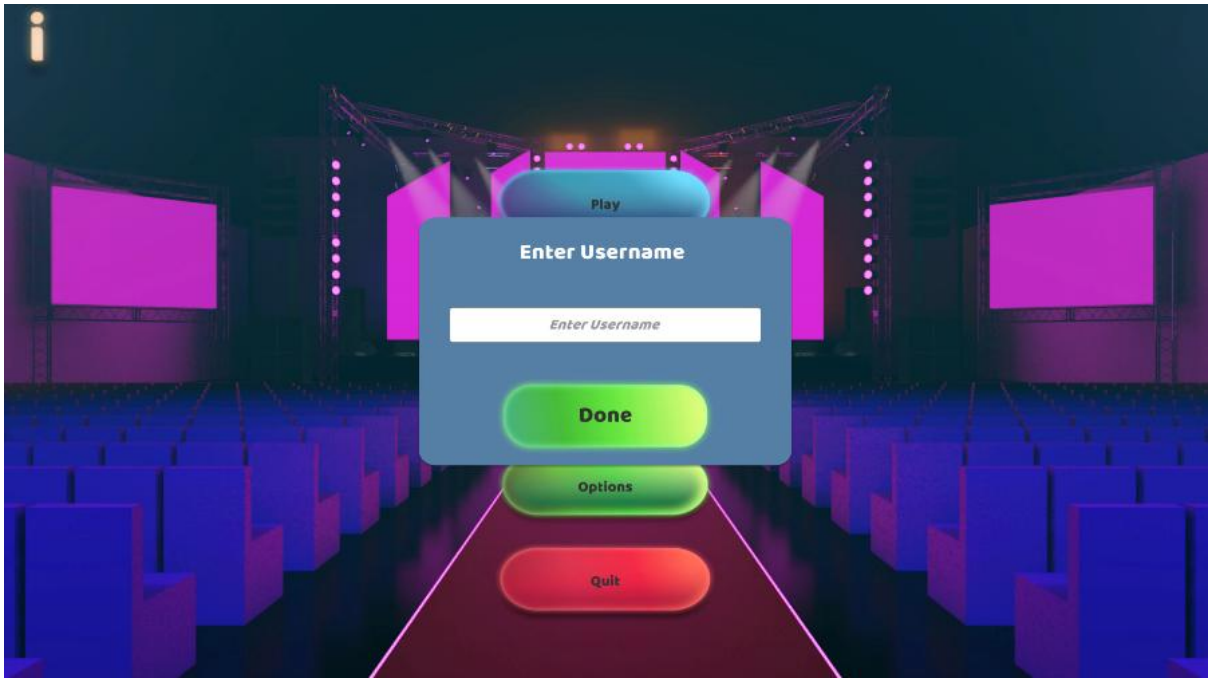
Freesound_community (exotonestudio) – open hat snake (miss sound). [Online] Available at https://pixabay.com/sv/sound-effects/musikalisk-open-hat-snake-100639/. Last accessed 02/05/2026

12. Appendices

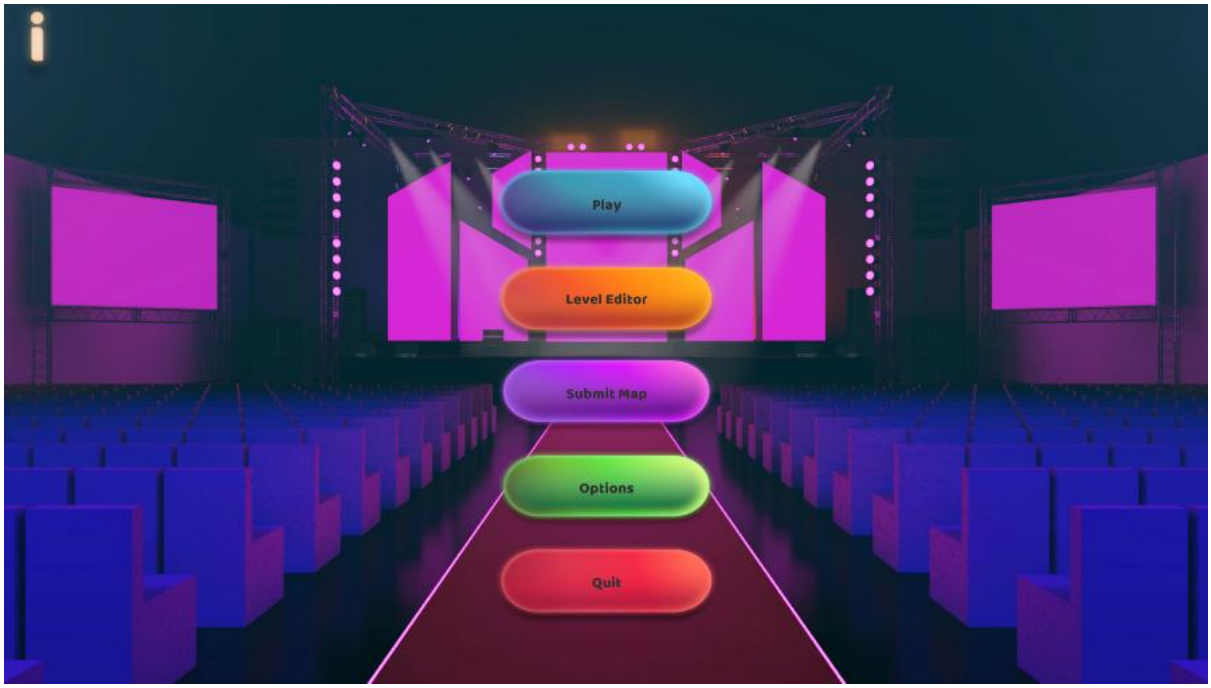
12.1 Appendix A – User Guide

To run the game download the zip file containing the executable from the GitHub repository Extract the folder and then run the exe file inside.

To setup the default level copy the folder within Assets/DefaultLevel called 'alexgrohl-energetic-action-sport-500409'. Navigate to AppData/LocalLow/DefaultCompany/COMP3000/Music and copy the folder into here. If this folder does not exist yet you can either create it yourself or run the game first and enter your username. AppData is a hidden folder so ensure you can view them by clicking on view -> show -> hidden files in file explorer or pressing Win+R on the keyboard and enter %appdata% then click on AppData in the address bar.



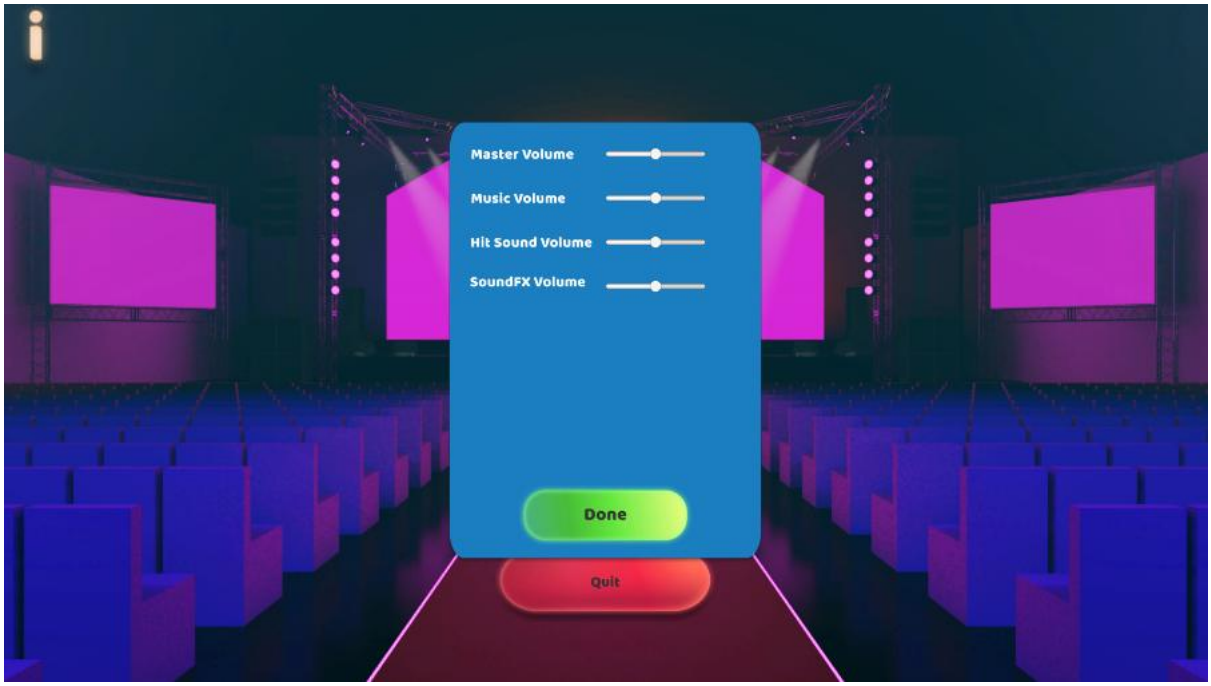
When you first load the game you are given the option to choose a username. Here You must click on the input field and type in what you would like to be called. Any issues with duplicate names or incorrect format will display underneath the input field.



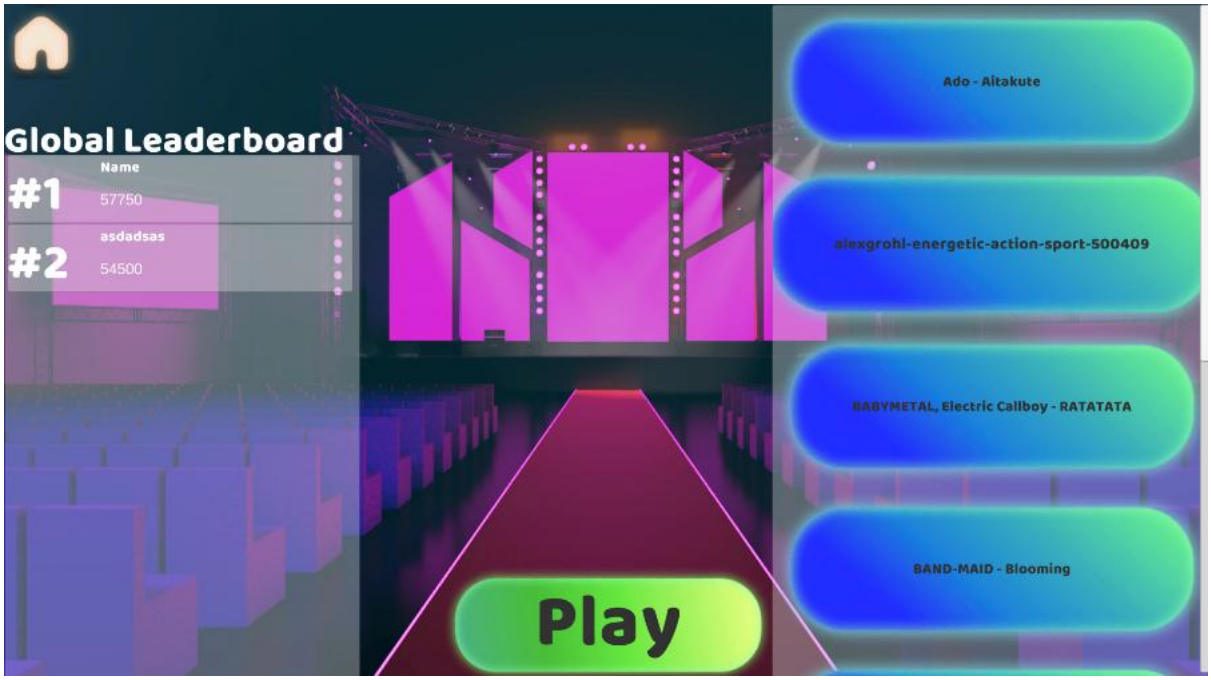
Navigation of the game UI is done through the mouse but when you play a level you must use a controller.

When you click on the options button in the main menu settings for audio are displayed. This includes master, music, soundFX and hit sounds. Master volume

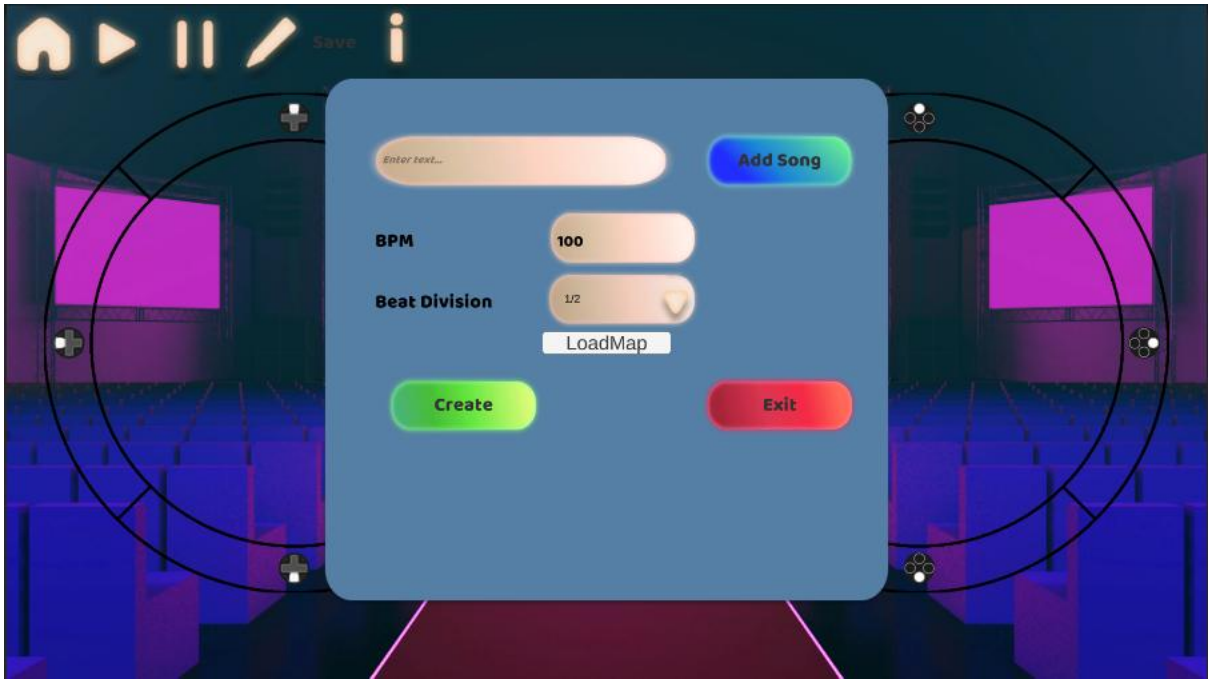
affects the volume of all sounds across the game, music affects the volume of the music played in the song select and the gameplay, soundFX adjusts the volume for button presses and hit sounds changes the sounds in the gameplay when you hit a note or if you miss a note.



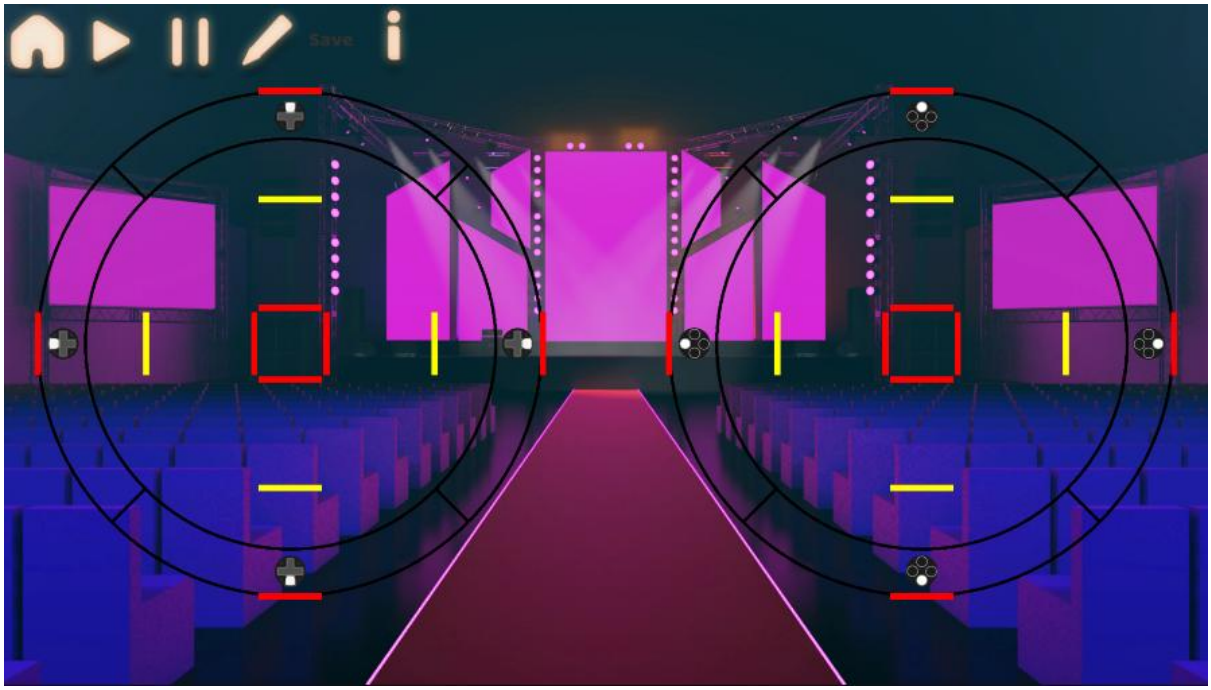
To choose a level to play click on the play button in the main menu. The scene that shows up next is the song select scene. Here there is a list on the right side of the screen with buttons containing the song name. If you click on one of these buttons the button will become larger to show it has been selected and the leaderboard on the left side of the screen will update every 0.5 seconds with the top 25 scores on that level. The song of the level you have just clicked on will start playing to give you a preview of the music. In order to play the level press the Play button in the bottom middle of the screen. You can also press the left or right arrows to move around up or down the list of songs you can play.



In the gameplay scene the two circles will show the buttons needed to press for each lane. This game was tested using an Xbox One controller so it cannot be confirmed that other controllers will work. The left circle uses the D-pad buttons and the right circle uses the ABXY buttons. A tap note is a note that just requires you to press the button down for the lane that it is in. A hold note is a note that requires you to press the button down when the head reaches the hit area and hold it. You must then release that button when the tail reaches the play area. The visual difference between a hold note and a tap note is that a hold note will have a black line connecting the head to the tail. The tail may not be visible when you have to hit the head but the line connecting them will still be seen and will stop in the centre of the circle. For a reminder of this information press the information button in the top left of the main menu.



To create a level in the level editor you must first click add song. File explorer will pop up and you must then choose an mp3 file to add. Next you must put in the beats per minute of the song. This can be found by searching for the song on a website like tunebat.com. Finally you must choose the beat division. This is how many timing lines will represent one beat. For example if you choose a division of ½ there would be two timing lines for each beat or if you choose ¼ there would be four timing lines for each beat.



To add a note you can click on a timing line and to destroy it click on the timing line again. To add hold notes press ‘h’ and click on one timing line to add the head. Then click on another timing line in the same lane on a timing line with a later timestamp to add the tail. To change back to adding tap notes simply press ‘h’ again on the keyboard.

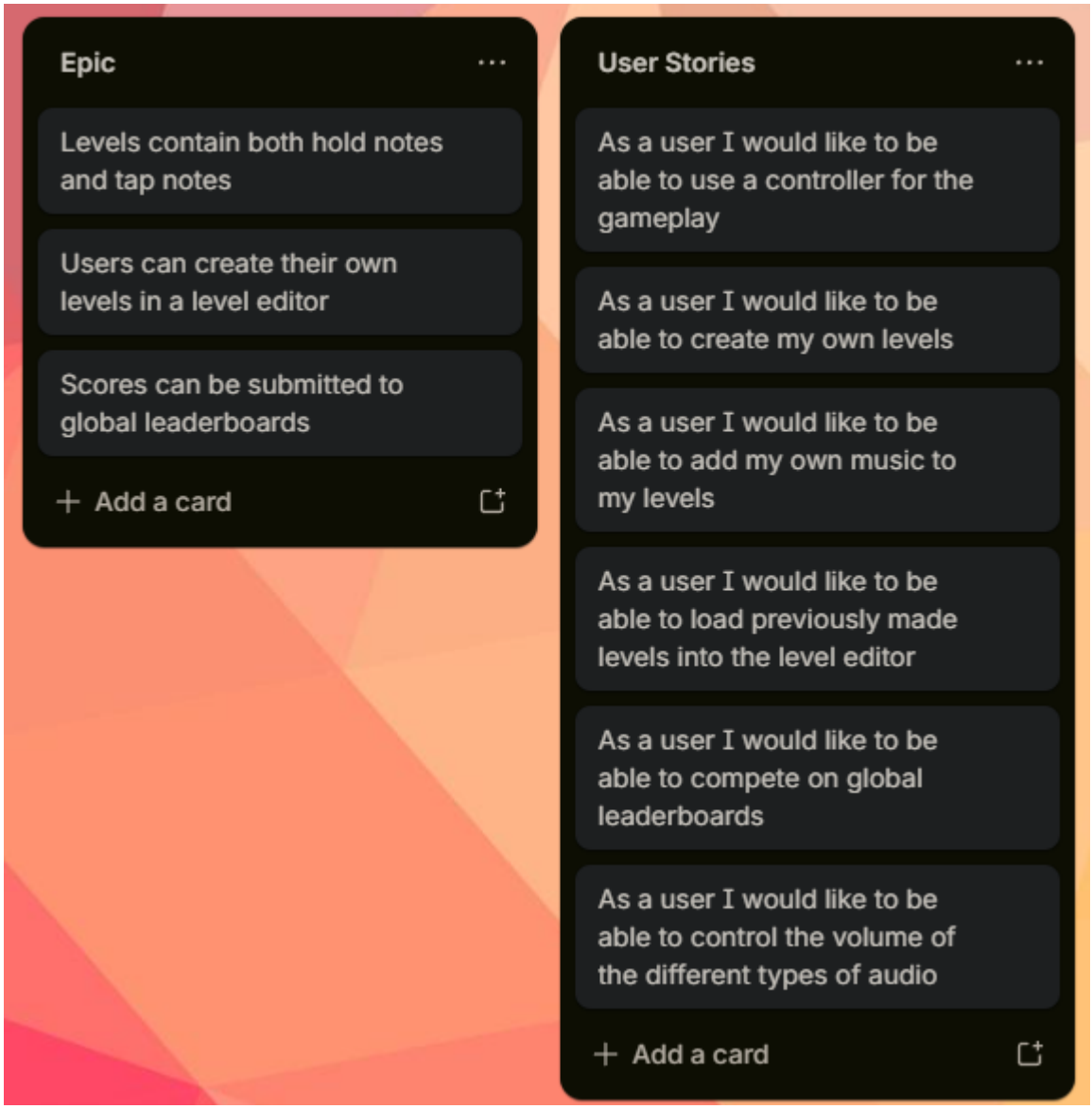
To place notes at later timings click the play/pause button at the top of the screen. You can also press the left and right arrow keys which will skip forward in the song or go slightly back.

To save a level press the save button found next to the play/pause button. The folder containing the song and the level file can be found in Appdata/LocalLow/DefaultComany/COMP3000/Music. Do not make any manual changes to the level file as it can cause the level to no longer be playable. Once a map has been finished it can be submitted to the leaderboard by clicking the submit button on the main menu.

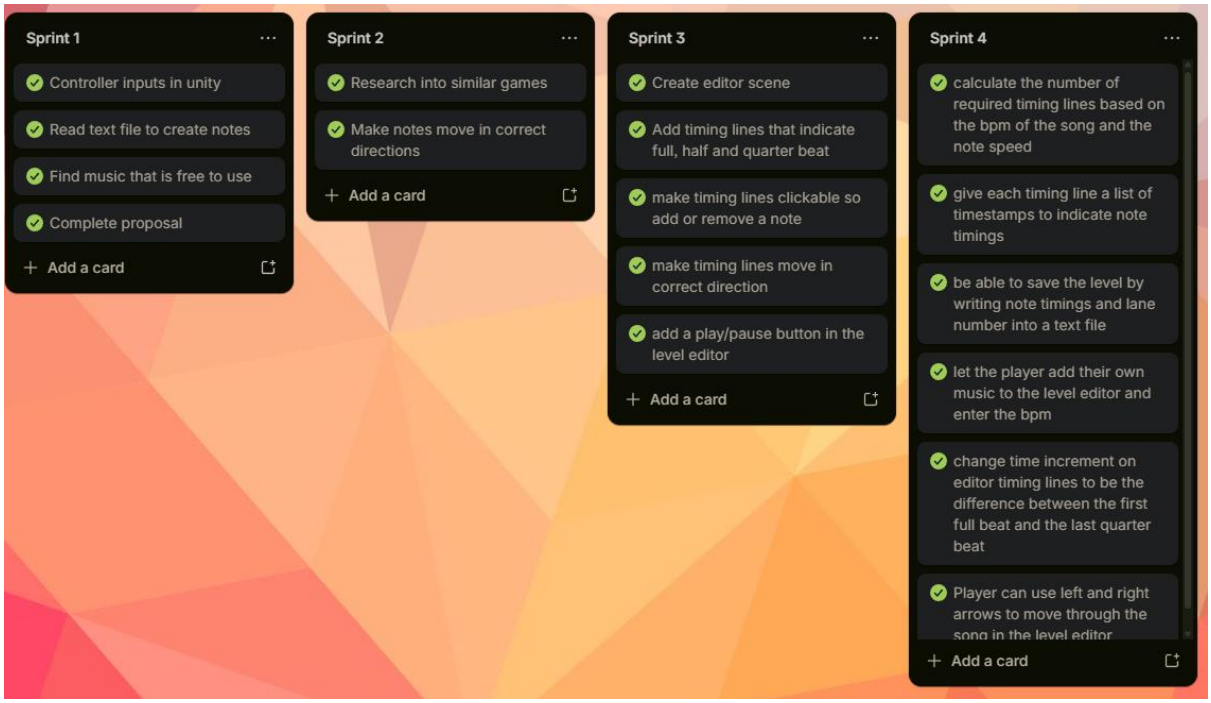
If you need a reminder on how the level editor works press the information button next to the save button which will contain all the information needed to know how it works.

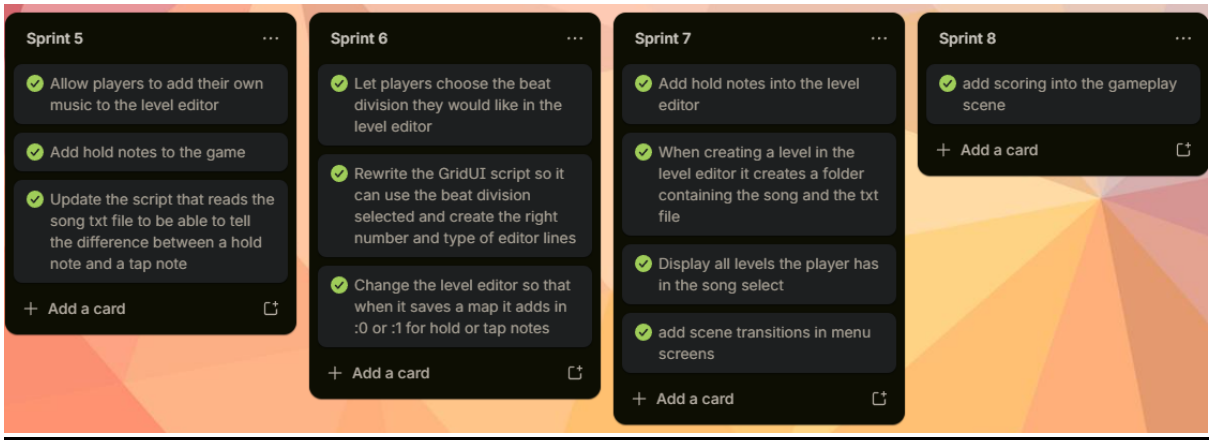
12.2 Appendix B – Trello Board

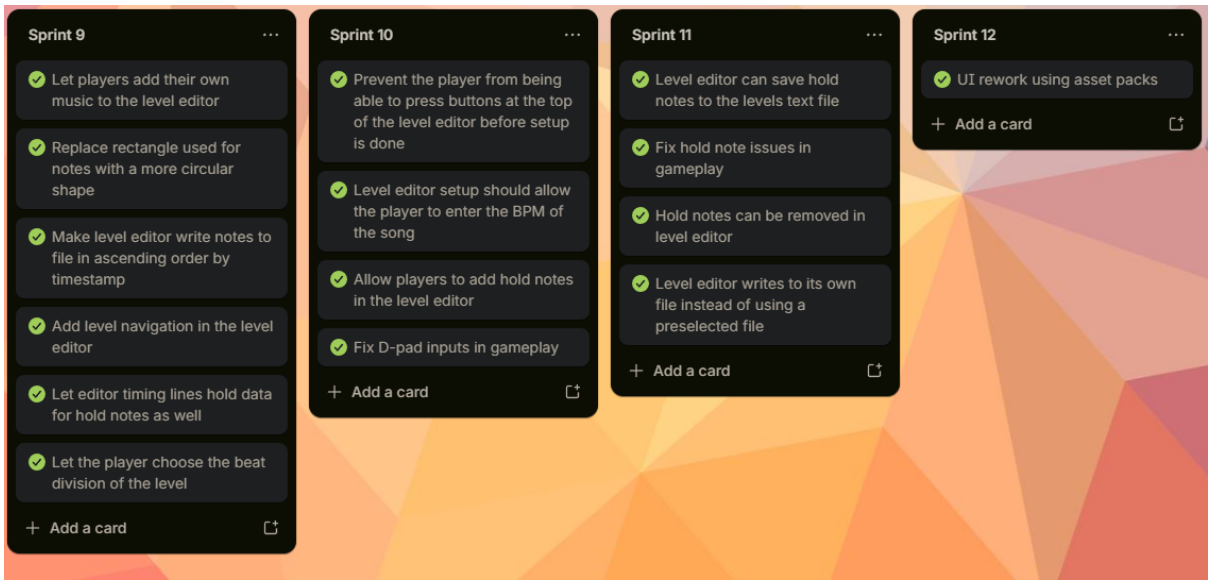
12.2.1 Epics and User Stories

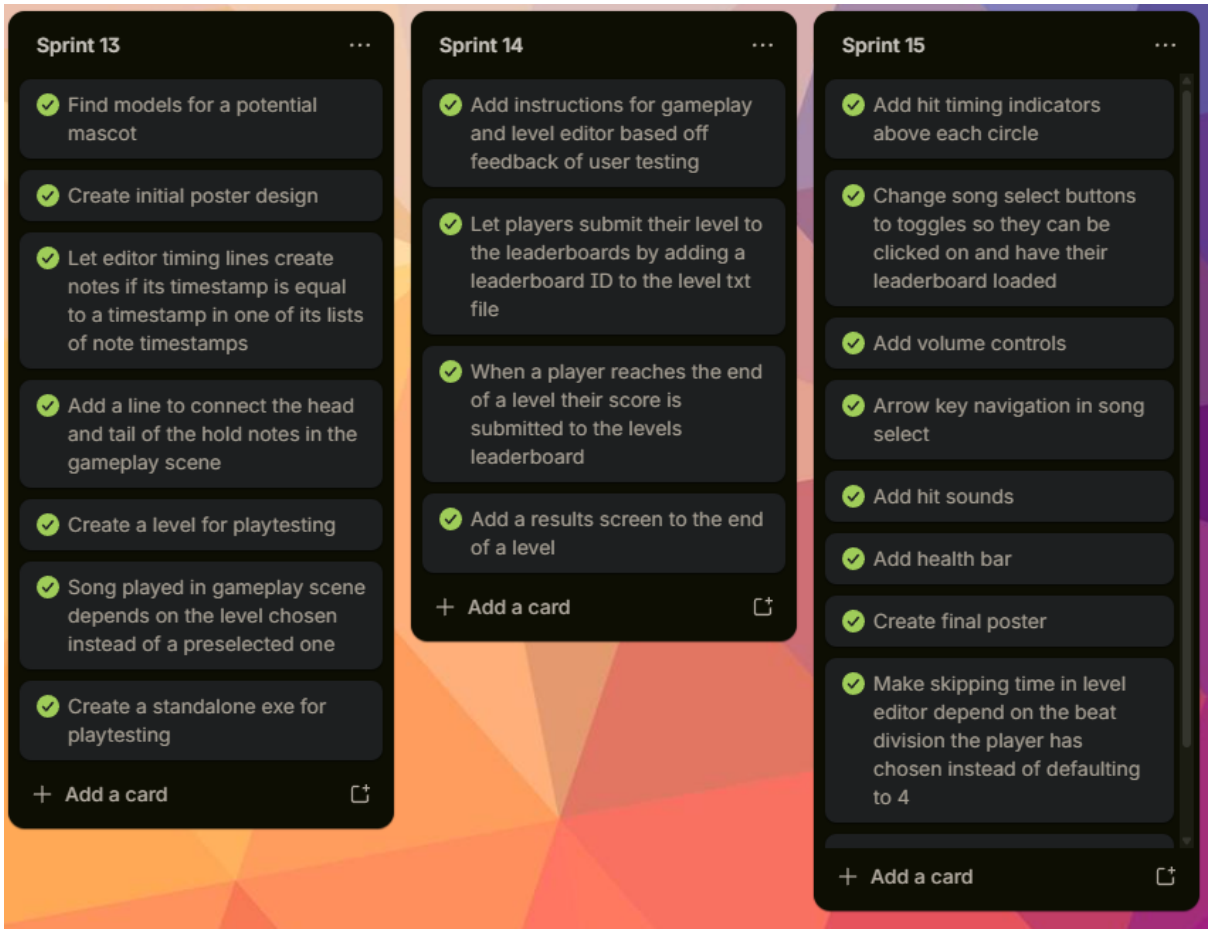

Epic
Levels contain both hold notes and tap notes
Users can create their own levels in a level editor
Scores can be submitted to global leaderboards
Add a card
User Stories
As a user I would like to be able to use a controller for the gameplay
As a user I would like to be able to create my own levels
As a user I would like to be able to add my own music to my levels
As a user I would like to be able to load previously made levels into the level editor
As a user I would like to be able to compete on global leaderboards
As a user I would like to be able to control the volume of the different types of audio
Add a card

12.2.2 Sprint Plans




Sprint 5
Allow players to add their own music to the level editor
Add hold notes to the game
Update the script that reads the song txt file to be able to tell the difference between a hold note and a tap note
Add a card
Sprint 6
Let players choose the beat division they would like in the level editor
Rewrite the GridUI script so it can use the beat division selected and create the right number and type of editor lines
Change the level editor so that when it saves a map it adds in :0 or :1 for hold or tap notes
Add a card
Sprint 7
Add hold notes into the level editor
When creating a level in the level editor it creates a folder containing the song and the txt file
Display all levels the player has in the song select
add scene transitions in menu screens
Add a card
Sprint 8
add scoring into the gameplay scene
Add a card


Sprint 9
Let players add their own music to the level editor
Replace rectangle used for notes with a more circular shape
Make level editor write notes to file in ascending order by timestamp
Add level navigation in the level editor
Let editor timing lines hold data for hold notes as well
Let the player choose the beat division of the level
Add a card
Sprint 10
Prevent the player from being able to press buttons at the top of the level editor before setup is done
Level editor setup should allow the player to enter the BPM of the song
Allow players to add hold notes in the level editor
Fix D-pad inputs in gameplay
Add a card
Sprint 11
Level editor can save hold notes to the levels text file
Fix hold note issues in gameplay
Hold notes can be removed in level editor
Level editor writes to its own file instead of using a preselected file
Add a card
Sprint 12
UI rework using asset packs
Add a card

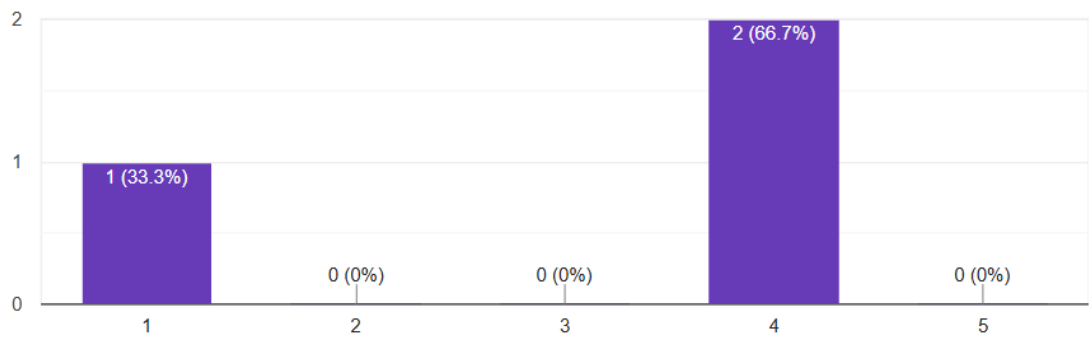


12.3 Appendix C - Testing Results

How easy was it to navigate the UI

Copy chart

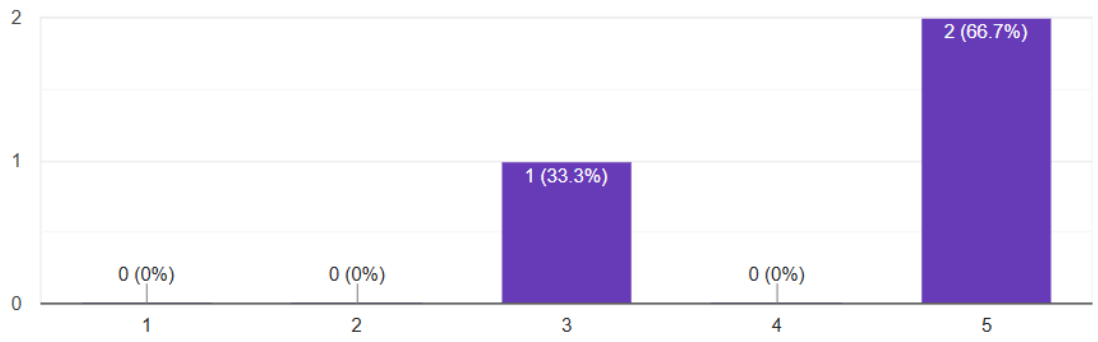
3 responses



How easy was it to understand what buttons to press when playing a level

Copy chart

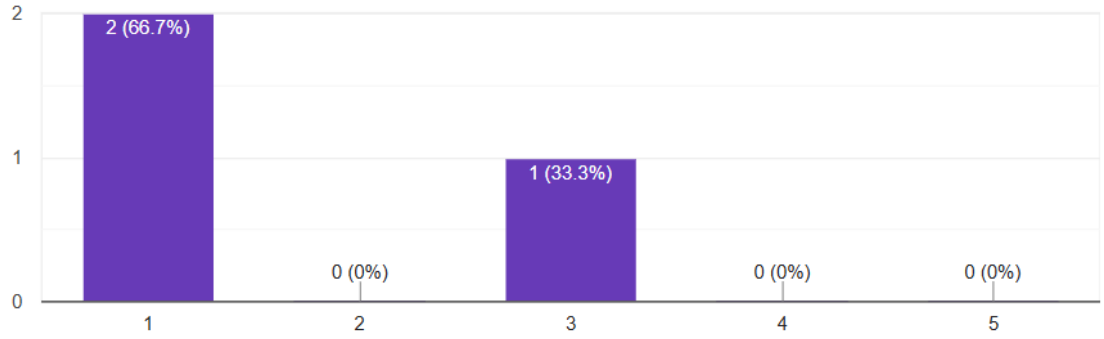
3 responses



How easy was it to understand what to do differently for a hold note and a tap note

Copy chart

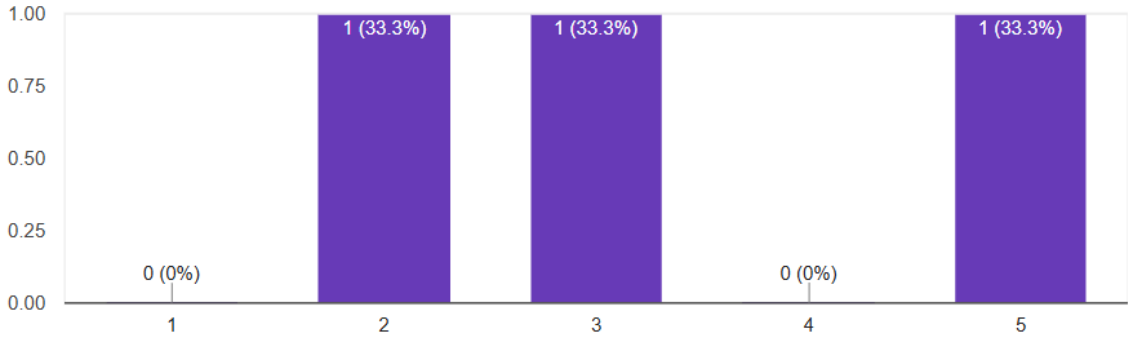
3 responses



When adding a song in the level editor did you understand what you had to do/what the fields meant

Copy chart

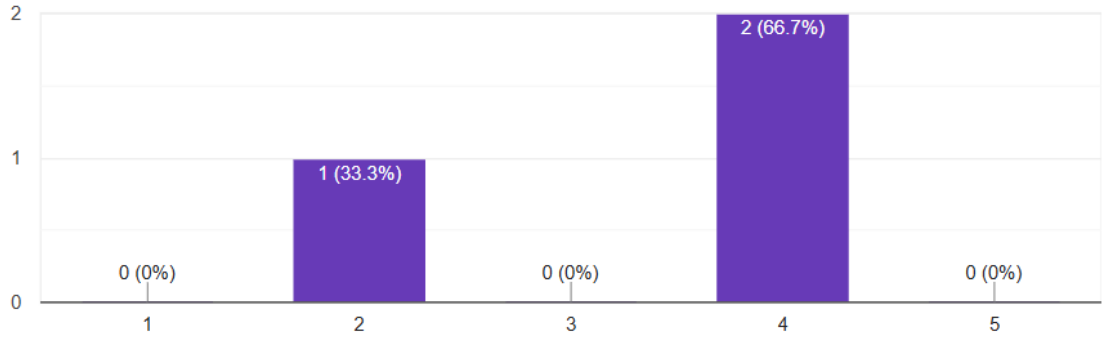
3 responses



How intuitive was it to use the level editor to create a level

Copy chart

3 responses



Do you have any suggestions to improve UI navigation

3 responses

nothing for the navigation

modernise the feel of it but otherwise its fine

Nothing bad with the UI

Do you have any suggestions to improve the level editor

3 responses

moving around the timeline of the level editor could be improved, as you have to be specific with how much you play and pause for.

somehow communicate the bars are clickable as designing a level is completely unexplained

More visual feedback when selecting note types, fix orientation of notes/validation to inputs

Do you have any suggestions to improve the gameplay

3 responses

Gameplay is good, just difficult.

make the notes different colours rather than just black

Visual feedback on successful/unsuccessful hits, add scoring?